

INTERNATIONAL POSTCODE SYSTEM (IPCS)

por

Bartosz Cieřlicki

Rochdale 2018

Premisas del sistema

El concepto principal del sistema es crear un sistema uniforme de c3digos postales para toda la Tierra. El objetivo m1s importante es:

CODIFICAR LAS COORDENADAS GEOGR1FICAS DE CADA C3DIGO POSTAL EN SU PROPIO REGISTRO.

El sistema as1 construido eliminar1 la necesidad de una gran base de datos global que contenga la ubicaci3n de cada c3digo postal. Tambi3n reducir1 o eliminar1 por completo los costes de geocodificaci3n, adem1s de acelerar notablemente los sistemas inform1ticos que buscan una posici3n geogr1fica para una direcci3n solicitada.

Premisas:

- El 1rea identificada por el c3digo postal no deber1a superar los 1000 m², de modo que se pueda llegar f1cil y precisamente al lugar solicitado usando un mapa o navegaci3n por sat3lite.
- El sistema debe cubrir cada parte del planeta, independientemente de la poblaci3n, la densidad de edificaci3n, el relieve o el tipo de superficie.
- Los c3digos postales deben calcularse / crearse mediante un algoritmo, y no asignarse arbitrariamente.
- A partir de cada registro de c3digo postal se pueden calcular las coordenadas geogr1ficas que indican el centro del 1rea.

El International Postcode System (IPCS):

- deber1a basarse en el alfabeto latino, el m1s extendido en el mundo.
- deber1a tener un n1mero relativamente reducido de caracteres, que permita memorizarlo f1cilmente.
- no deber1a contener caracteres especiales propios de un pa1s concreto.
- deber1a eliminar los errores m1s frecuentes de escritura y lectura (es decir, la letra O confundida con el n1mero 0, y la letra I confundida con el n1mero 1).

Construcción de los sectores

Para calcular el International Postcode System (IPCS) utilizamos la latitud y la longitud en grados decimales. La longitud se sitúa en el rango $(-180, 180>$, mientras que la latitud se sitúa en el rango $(-90, 90>$. El primer paso, muy importante, para calcular el código postal internacional consiste en redondear la latitud y la longitud a un cuarto de 0,001.

Ejemplo:

Supongamos que el número $t \in (0.000, 0.001>$, siendo x nuestro redondeo. Si:

$$x \in (0.000, 0.000125) \rightarrow x = 0.000$$

$$x \in (0.000125, 0.000375) \rightarrow x = 0.00025$$

$$x \in (0.000375, 0.000625) \rightarrow x = 0.0005$$

$$x \in (0.000625, 0.000875) \rightarrow x = 0.00075$$

$$x \in (0.000875, 0.001> \rightarrow x = 0.001$$

Gracias al uso de estos redondeos, toda la Tierra queda dividida en zonas. Las coordenadas obtenidas identifican cada sector creado en la Tierra y, al mismo tiempo, son su centro. Esto cumple la premisa planteada al principio.

El extracto de la cuadrícula tiene el siguiente aspecto:

La estructura de los sectores, basada en las coordenadas, hace que la superficie de cada sector, formado al redondear la latitud y la longitud, disminuya a medida que nos acercamos a los polos de la Tierra. Las mayores superficies estarán, por tanto, en el ecuador y rondarán los 728 m², lo que cumple nuestra premisa.

Podemos calcular el número de sectores creados de esta manera. Los números se redondean a un cuarto de milésima, por lo que cada milésima se divide en 4.

Para la longitud tenemos el rango numérico $(-180, 180>$. Con x indicamos el número de divisiones. Es igual a:

$$x = 360 * 1000 * 4 \quad (1)$$

Para la latitud tenemos el rango numérico $(-90, 90>$. Con y indicamos el número de divisiones. Es igual a:

$$y = 181 * 1000 * 4 \quad (2)$$

(El valor 181, en lugar de 180, tiene en cuenta que el rango de latitud $(-90, 90>$ es cerrado por ambos extremos —el polo Sur (-90°) y el polo Norte (90°) son dos puntos distintos—, a diferencia del rango de longitud $(-180, 180>$, que solo es cerrado por un extremo, ya que -180° y 180° designan el mismo meridiano.)

Usando (1) y (2), calculamos el número de sectores para toda la Tierra:

$$\text{Number of sectors} = x * y = (360 * 1000 * 4) * (181 * 1000 * 4) = 1\,042\,560\,000\,000 \quad (3)$$

Solo queda construir el Código Postal Internacional (IPCS) para cada uno de los sectores formados/creados.

Construcción del International Postcode System (IPCS)

Usando letras latinas y números, disponemos de 10 dígitos y 26 letras. Eliminamos del alfabeto las letras O e I (para cumplir las premisas del sistema). Así obtenemos los siguientes 34 caracteres: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, J, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z}. A partir de estos 34 caracteres creamos un sistema numérico: un sistema en base 34. A la izquierda tenemos los números en el sistema en base 34; a la derecha, su equivalente en el sistema decimal.

0 = 0	8 = 8	G = 16	Q = 24	Y = 32
1 = 1	9 = 9	H = 17	R = 25	Z = 33
2 = 2	A = 10	J = 18	S = 26	
3 = 3	B = 11	K = 19	T = 27	
4 = 4	C = 12	L = 20	U = 28	
5 = 5	D = 13	M = 21	V = 29	
6 = 6	E = 14	N = 22	W = 30	
7 = 7	F = 15	P = 23	X = 31	

Cada número en base 34 puede representarse en el sistema decimal de la siguiente manera:

$$\dots dcba_{34} = f(a) \times 34^0 + f(b) \times 34^1 + f(c) \times 34^2 + f(d) \times 34^3 + \dots$$

, donde f es una función que asigna a cada dígito del sistema en base 34 su equivalente en el sistema decimal (véase (4)). Aquí, a, b, c, d son dígitos del sistema en base 34.

Ejemplo:

$$\begin{aligned} AGZ15Z_{34} &= f(Z) \times 34^0 + f(5) \times 34^1 + f(1) \times 34^2 + f(Z) \times 34^3 + f(G) \times 34^4 + f(A) \times 34^5 \\ &= 33 \times 34^0 + 5 \times 34^1 + 1 \times 34^2 + 33 \times 34^3 + 16 \times 34^4 + 10 \times 34^5 \\ &= 33 \times 1 + 5 \times 34 + 1 \times 1156 + 33 \times 39304 + 16 \times 1336336 + 10 \times 45435424 \\ &= 33 + 170 + 1156 + 1297032 + 21381376 + 454354240 = 477034007 \end{aligned}$$

Como se puede ver, la notación en base 34 es mucho más corta. Necesitaremos 8 caracteres para generar códigos postales para todo el planeta. El número máximo de caracteres en el sistema postal inglés e irlandés actual es 7, en Japón 7 dígitos, en los Países Bajos 6 caracteres, etc. Teniendo en cuenta la longitud de los códigos postales en otros países, y el hecho de que el International Postcode System es un sistema global, asumimos que 8 caracteres son aceptables y cumplen las premisas de nuestro sistema.

El número más grande que podemos almacenar con 8 dígitos en nuestro sistema en base 34 es (se han omitido los cálculos detallados):

$$ZZZZZZZZ_{34} = 1\,785\,793\,904\,895$$

De ello se deduce que la Tierra podría tener 1 785 793 904 896 códigos postales (este número es una unidad mayor que el máximo, porque 00000000 también puede ser un código). Por el cálculo anterior (3) sabemos que el número de sectores de nuestra estructura es 1 042 560 000 000. Este número es menor que 1 785 793 904 895, por lo que 8 dígitos en el sistema en base 34 bastan para identificar unívocamente cada sector. Las áreas de los sectores no son iguales debido a esta estructura. Las mayores se sitúan cerca del ecuador y son $\approx 728 \text{ m}^2$. La superficie de la Tierra es de unos

$510\,100\,000\text{ km}^2 = 510\,100\,000\,000\,000\text{ m}^2$. Para calcular el área media de un sector, realizamos una simple operación:

$$510\,100\,000\,000\,000\text{ m}^2 / 1\,042\,560\,000\,000 \approx 489.2764\text{ m}^2$$

Así pues, $ZZZZZZZZ_{34}$ cumple nuestras premisas, y el área de cada sector es menor que 1000 m^2 .

Ahora solo queda convertir el centro del sector al International Postcode System (IPCS).

Para nuestros fines, representamos la parte entera de los números negativos de la siguiente manera:

- Para la longitud, la parte entera negativa x se convierte en $360 + x$.
- Para la latitud, el entero negativo y se convierte en $180 + y$.

La parte decimal permanece sin cambios. En resumen:

$$\text{Longitude } -XXX.xxxxx \rightarrow (360 + (-XXX)).xxxxx$$

$$\text{Latitude } -YYY.yyyyy \rightarrow (180 + (-YYY)).yyyyy$$

Ejemplos de la conversión:

- El número -75.43725 para la longitud pasa a ser 285.43725 , porque $360 + (-75) = 285$.
- El número -0.411250 para la longitud pasa a ser 360.411250 , porque $360 + 0 = 360$.
- El número -15.44375 para la latitud pasa a ser 165.44375 , porque $180 + (-15) = 165$.

Al crear el algoritmo debemos tener en cuenta que, en este registro, el polo Sur a 90 grados de latitud y el polo Norte a -90 grados de latitud se consideran equivalentes.

Teniendo en cuenta la conversión de números negativos y el redondeo, la forma general de las coordenadas registradas en el sistema decimal es:

$$\text{Longitude: } XXX.xxxqq, \text{ where } XXX \in \{x \in \mathbb{N}: 000 \leq x \leq 360\}, xxx \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}, qq \in \{00, 25, 50, 75\}$$

$$\text{Latitude: } YYY.yyygg, \text{ where } YYY \in \{x \in \mathbb{N}: 000 \leq x \leq 180\}, yyy \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}, gg \in \{00, 25, 50, 75\}$$

En el siguiente paso, cada elemento de los números $XXX.xxxqq$ y $YYY.yyygg$ se convierte a sistema binario.

$$XXX_{10} = XXXXXXXXX_2 \text{ — max. } 360, \text{ requires } 9 \text{ bits.}$$

$$xxx_{10} = xxxxxxxxx_2 \text{ — max. } 999, \text{ requires } 10 \text{ bits.}$$

$$qq_{10} = qq_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

$$YYY_{10} = YYYYYYYY_2 \text{ — max. } 180, \text{ requires } 8 \text{ bits.}$$

$$yyy_{10} = yyyyyyyyy_2 \text{ — max. } 999, \text{ requires } 10 \text{ bits.}$$

$$gg_{10} = gg_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

A continuación, disponemos los números binarios en el siguiente orden:

$$XXXXXXXXXX YYYYYYYY xxxxxxxxxx yyyyyyyyyy qq gg_2$$

Tomamos un número de al menos 41 bits, lo ponemos a cero e introducimos los bits anteriores de derecha a izquierda, manteniendo su orden. El número obtenido es igual a:

$$XXXXXXXXXXYYYYYYYxxxxxxxxxyyyyyyyyyqqgg_2$$

En el último paso, convertimos el número binario a base 34. El número en base 34 obtenido es también nuestro Código Postal Internacional (IPCS). Las coordenadas del centro del sector quedan codificadas en nuestro registro del Código Postal Internacional (IPCS).

El orden en que se escriben los bits es muy importante, porque, redondeada y presentada según nuestra convención, la coordenada con el mayor valor numérico es 360.99975 , 180.99975 . Al convertirla a

sistema binario obtenemos el número:

$$10110100010110100111110011111110011111112 = 1\,549\,224\,509\,055_{10}$$

El número obtenido es menor que el número máximo del código en base 34, que es $ZZZZZZZ_{34} = 1\,785\,793\,904\,895_{10}$. Cambiar el orden de los bits podría superar ese valor máximo, lo que haría imposible crear códigos postales para todos los sectores.

Para obtener las coordenadas del centro del sector (Código Postal Internacional), realizamos la operación en orden inverso.

Ejemplos:

Construimos el Código Postal Internacional (IPCS) para las coordenadas -1.347120, 53.983489 de la siguiente manera:

Redondeamos, convertimos las coordenadas y obtenemos 359.34700, 53.98350

Convertimos cada elemento de las coordenadas —es decir, los números 359, 347, 00, 53, 983, 50— a binario como se describió anteriormente, y obtenemos: 101100111_2 , 0101011011_2 , 00_2 , 01101010_2 , 1111010111_2 , 10_2

Disponemos los números en el orden correcto: 101100111_2 , 0011010_{21} , 0101011011_2 , 1111010111_2 , 00_2 , 10_2

Formamos el número $10110011100110101010101101111110101110010_2$

Convertimos el número binario a base 34 y obtenemos $VCPM6TKY_{34}$

El número obtenido es el Código Postal Internacional (IPCS): VCPM 6TKY

Las coordenadas del Código Postal Internacional 1GL8 YT7F se calculan de la siguiente manera:

Convertimos el número en base 34 $1GL8YT7F_{34}$ a binario y obtenemos $000010010001100101100000011010011011010012$

Extraemos los números 00001001_2 , 0011001_{20} , 1100000011_2 , 0100110110_2 , 10_2 , 01_2

Cambiamos el orden a 00001001_2 1100000011_2 , 10_2 , 0011001_{20} 0100110110_2 , 01_2

Convertimos los números binarios a decimal: 18, 771, 50, 50, 310, 25

A partir de los números obtenidos, obtenemos las coordenadas 18.77150, 50.31025

Función que genera y calcula coordenadas a partir del Código Postal Internacional

A continuación se muestran dos funciones escritas en C#. La primera construye el Código Postal Internacional, y la segunda calcula las coordenadas a partir del Código Postal Internacional.

```
public string GeneratePostcode(double Lon, double Lat)
{
    if ((Lon > 180) || (Lon < -180) || (Lat > 90) || (Lat < -90)) return "";
    if (Lat > 89.99975) Lat = 89.99975;
    if (Lat < -89.99975) Lat = -89.99975;

    string[] Coder = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", "F", "G", "H", "J",
        "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z" };

    if (Lat < 0)
    {
        Lat = ((long)Lat + 180) + Math.Abs(Lat - (long)Lat);
    }
    if (Lon < 0)
    {
        Lon = ((long)Lon + 360) + Math.Abs(Lon - (long)Lon);
    }

    string PartPostcodeLat = Lat.ToString("000.000000");
    string PartPostcodeLon = Lon.ToString("000.000000");

    try
    {
        string[] StrsLat = PartPostcodeLat.Split('.');
        string[] StrsLon = PartPostcodeLon.Split('.');

        ulong OptionsLat = 5;
        long Precision = Convert.ToInt32(StrsLat[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLat = 0;
        }
        else if (Precision < 375)
        {
            OptionsLat = 1;
        }
        else if (Precision < 625)
        {
            OptionsLat = 2;
        }
        else if (Precision < 875)
        {
            OptionsLat = 3;
        }

        ulong TempHiLat = Convert.ToUInt64(StrsLat[0]);
        TempHiLat = TempHiLat << 24;
        ulong TempLoLat = 0;
        if (StrsLat.Length > 1)
        {
            TempLoLat = Convert.ToUInt64(StrsLat[1].Substring(0, 3));
            if (OptionsLat == 5)
            {
                TempLoLat++;
            }
            else
            {
                {

```

```

        TempHiLat = TempHiLat | OptionsLat;
    }

    TempLoLat = TempLoLat << 4;
}

ulong OptionsLon = 5;
Precision = Convert.ToInt32(StrsLon[1].Substring(3));
if (Precision < 125)
{
    OptionsLon = 0;
}
else if (Precision < 375)
{
    OptionsLon = 1;
}
else if (Precision < 625)
{
    OptionsLon = 2;
}
else if (Precision < 875)
{
    OptionsLon = 3;
}

ulong TempHiLon = Convert.ToUInt64(StrsLon[0]);
TempHiLon = TempHiLon << 32;
ulong TempLoLo = 0;
if (StrsLon.Length > 1)
{
    TempLoLo = Convert.ToUInt64(StrsLon[1].Substring(0, 3));
    if (OptionsLon == 5)
    {
        TempLoLo++;
    }
    else
    {
        TempHiLon = TempHiLon | (OptionsLon * 4);
    }
}

TempLoLo = TempLoLo << 14;
}

ulong UlongResHi = TempHiLat | TempHiLon | TempLoLat | TempLoLo;

ulong Pos = 0;
string Res = "";
for (int i = 7; i >= 0; i--)
{
    Pos = UlongResHi / (ulong)Math.Pow(34, i);
    UlongResHi -= (Pos * (ulong)Math.Pow(34, i));
    Res += Coder[Pos];
}

return Res;
}
catch
{
    return "";
}
}

public bool ReversePostcode(string Postcode, out double Lat, out double Lon)
{
    if (Postcode.Length == 8)
    {

```

```

Postcode = Postcode.ToUpper();
string Coder = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
ulong Hi = 0;

for (int i = 0; i < 8; i++)
{
    Hi += (ulong)Coder.IndexOf(Postcode[i]) * (ulong)Math.Pow(34, (7 - i));
}

ulong OptionLat = (Hi & 3);
double HalfLat = 0;
switch (OptionLat)
{
    case 1:
        HalfLat = 0.00025;
        break;
    case 2:
        HalfLat = 0.0005;
        break;
    case 3:
        HalfLat = 0.00075;
        break;
}

ulong OptionLon = (Hi & 12) >> 2;
double HalfLon = 0;
switch (OptionLon)
{
    case 1:
        HalfLon = 0.00025;
        break;
    case 2:
        HalfLon = 0.0005;
        break;
    case 3:
        HalfLon = 0.00075;
        break;
}

long LoLat = ((long)Hi >> 4) & 1023;
long HiLat = ((long)Hi >> 24) & 255;
if (HiLat > 180)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLat > 90)
{
    HiLat = HiLat - 180;
    Lat = HiLat - ((LoLat / 1000.0) + HalfLat);
}
else
{
    Lat = HiLat + ((LoLat / 1000.0) + HalfLat);
}

long LoLon = ((long)Hi >> 14) & 1023;
long HiLon = ((long)Hi >> 32);
if (HiLon > 360)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLon > 180)

```

```
{
    HiLon = HiLon - 360;
    Lon = HiLon - ((LoLon / 1000.0) + HalfLon);
}
else
{
    Lon = HiLon + ((LoLon / 1000.0) + HalfLon);
}

return true;
}
else
{
    Lat = 0;
    Lon = 0;
    return false;
}
}
```